

CS5013 Design Document

Hostel Cycle Booking Web Portal

Yashas Katyal, CE24B128 Aditya Gautam, CE24B001

11 September 2026

1 Scoping feedback and how it shaped this design

In the proposal review, course staff asked us to justify why the project needed the full semester given AI-assisted coding. In response we made the scope concrete rather than just arguing for it in words: we committed to ID-card verification (an OpenCV/Tesseract OCR pipeline) tied to institute smail login, live enforcement of the hostel's own booking rules (24-hour hold cap, 3 rentals/week), and a self-trained classifier that turns free-text return comments into an actionable maintenance list. Those four pieces are the actual sources of complexity in the system, and the architecture below is organized around them as separate modules so that each can be built, tested, and demoed independently rather than as one large opaque booking app.

2 Architecture note: modules and interfaces

The system is a single Spring Boot application, internally split into eight modules. Each module exposes a narrow Java interface (a Spring service bean) that the others call – there is no direct cross-module database access, so a module's internals can change without breaking its callers.

1. **Auth Module** – institute smail login via Spring Security + OAuth2.
 - `AuthService.login(oauthToken) -> ResidentSession`
 - `AuthService.currentResident(session) -> ResidentIdentity{rollNumber, smailAddress}`
2. **ID Verification Module** – OpenCV image preprocessing + Tesseract OCR (via Tess4j) on the resident's ID card photo.
 - `IdVerificationService.scanCard(imageBytes) -> ExtractedIdentity{name, rollNumber, hostel, confidence}`
 - `IdVerificationService.matchesLogin(ExtractedIdentity, ResidentIdentity) -> boolean`
3. **Rule Engine** – the hostel's own booking rules, independent of the booking flow itself so the limits can be unit-tested in isolation.
 - `RuleEngine.checkEligibility(residentId) -> EligibilityResult{allowed, reason}`
4. **Booking Module** – availability, reservation, and concurrency control over the cycle pool.
 - `BookingService.getAvailability() -> List<CycleStatus>`

- `BookingService.createBooking(residentId, cycleId, ExtractedIdentity) -> Booking`
– calls `IdVerificationService.matchesLogin` and `RuleEngine.checkEligibility` before committing
5. **Pickup/Return Module** – the guard-witnessed passcode confirmation at the cycle stand.
 - `TransactionService.confirmPickup(bookingId, passcode) -> CycleState`
 - `TransactionService.confirmReturn(bookingId, passcode, conditionNote) -> CycleState`
 6. **Maintenance Comment Module** – the post-return free-text box and its classifier.
 - `MaintenanceService.submitComment(bookingId, text) -> List<MaintenanceFlag>` – calls `CommentClassifier.classify`
 - `CommentClassifier.classify(text) -> List<String>` (Apache OpenNLP, trained on our labeled phrase set)
 7. **Reporting Module** – the GS/Warden-facing weekly view.
 - `ReportService.getWeeklyUsageReport() -> UsageReport`
 - `ReportService.getMaintenanceList() -> List<MaintenanceFlag>`
 8. **Persistence Module** – Spring Data JPA repositories over SQLite (`ResidentRepository`, `CycleRepository`, `BookingRepository`, `CommentRepository`), used by every module above but never bypassed.

3 Test plan

At least one test per module, run with JUnit 5 and Spring Boot's test slices:

- **Auth:** a request to a protected endpoint with no mail session is rejected with 401; a valid OAuth token resolves to the correct `ResidentIdentity`.
- **ID Verification:** a known-good sample ID photo yields the expected roll number and a confidence above threshold; a deliberately blurred/glare photo yields low confidence rather than a wrong-but-confident match.
- **Rule Engine:** a resident with 3 bookings already this rolling week is denied a 4th; a resident holding a cycle for 23 hours is eligible to return normally, at 25 hours the booking is flagged overdue.
- **Booking:** a concurrency test – two threads calling `createBooking` for the same cycle at the same time – asserts exactly one succeeds and the other receives a clear "already booked" result, not a corrupted record.
- **Pickup/Return:** an incorrect passcode leaves the cycle's state unchanged; the correct passcode transitions `Booked -> Issued` and, later, `Issued -> Returned`.
- **Maintenance Comment:** the comment "chain keeps slipping off" is classified with a chain-related flag; "cycle is totally fine" produces no flags.
- **Reporting:** after seeding a known set of bookings and returns in a test database, `getWeeklyUsageReport` returns the exact expected total usage hours per cycle.

- **Persistence:** a round-trip test – save a `Booking` entity, fetch it back by id, assert every field matches – against an in-memory test database.

4 Milestone plan (revised)

The proposal’s three-checkpoint plan (design doc / mid-demo / final) still holds, but each checkpoint is now defined against the eight modules above instead of a single ”booking workflow,” since that is what actually changed in response to the scoping feedback – we’re building four extra pieces of real infrastructure (OCR, smail cross-check, rule enforcement, comment classifier), not padding the original plan.

- **By the design doc (11 Sep 2026, this document):** Persistence Module’s schema finalized; Auth and Booking Modules scaffolded with static/prototype pages; ID Verification Module has a working OCR prototype tested against sample card photos outside the app.
- **By the mid-demo (9 Oct 2026):** Auth, ID Verification, Rule Engine, Booking, and Pickup/Return Modules integrated end-to-end – a resident can log in with smail, get OCR-verified, book a cycle under the 24-hour/3-per-week rules, and confirm pickup/return via passcode in front of the guard. Reporting Module shows raw usage hours.
- **By the final submission (6 Nov 2026):** Maintenance Comment Module (comment box + trained classifier) integrated; Reporting Module extended to combine guard notes and auto-flagged comment terms; the GS and guard can run the full system themselves on the hostel’s own infrastructure.

5 Risks and plan B

1. **Risk:** a resident’s phone can’t get a clean-enough photo of their ID card for OCR to read reliably (poor lighting, glare, a worn card), blocking them from booking. **Plan B:** allow manual roll-number entry after a fixed number of failed scan attempts, still cross-checked against the resident database and smail login.
2. **Risk:** setting up institute smail OAuth login may need IT/domain approval we don’t control on this timeline. **Plan B:** fall back to a simple smail-address-plus-OTP login instead of full OAuth if institute OAuth access isn’t available in time.
3. **Risk:** our self-trained maintenance-comment classifier may miss real issues or flag false positives on short, informal, or mixed-language comments. **Plan B:** always show the GS the raw comment alongside any auto-flagged terms, so the model is a triage aid rather than the sole source of truth.
4. **Risk:** the 24-hour/3-per-week rule enforcement has edge cases (e.g., a cycle stuck with a resident past 24 hours due to a broken lock, not their fault) that could unfairly count against them. **Plan B:** give the GS a manual override to waive or reset a limit case-by-case.